

Qualidade e Confiabilidade de **Software**

Prof. Esp.
Rita de Cássia da Costa Silva



QUALIDADE DE SOFTWARE



Qualidade de software é o grau em que um sistema, componente ou processo atende aos requisitos especificados e às expectativas do usuário.

Em outras palavras: um software de qualidade é útil, confiável, eficiente e fácil de manter.

Qualidade Interna, Externa e Em Uso



Qualidade **Interna**

Relacionada ao código-fonte, arquitetura e práticas de desenvolvimento.
Exemplo: modularidade, ausência de código duplicado, legibilidade.

Qualidade **Externa**

O comportamento do software quando executado.
Exemplo: desempenho, consumo de memória, ausência de falhas.

Qualidade **Em Uso**

A experiência do usuário final.
Exemplo: facilidade de uso, produtividade, satisfação.

Normas e Padrões de Qualidade

A norma ISO/IEC 25010:2011 define o modelo de qualidade de software mais utilizado no mundo.

Ela substituiu o antigo padrão ISO/IEC 9126.

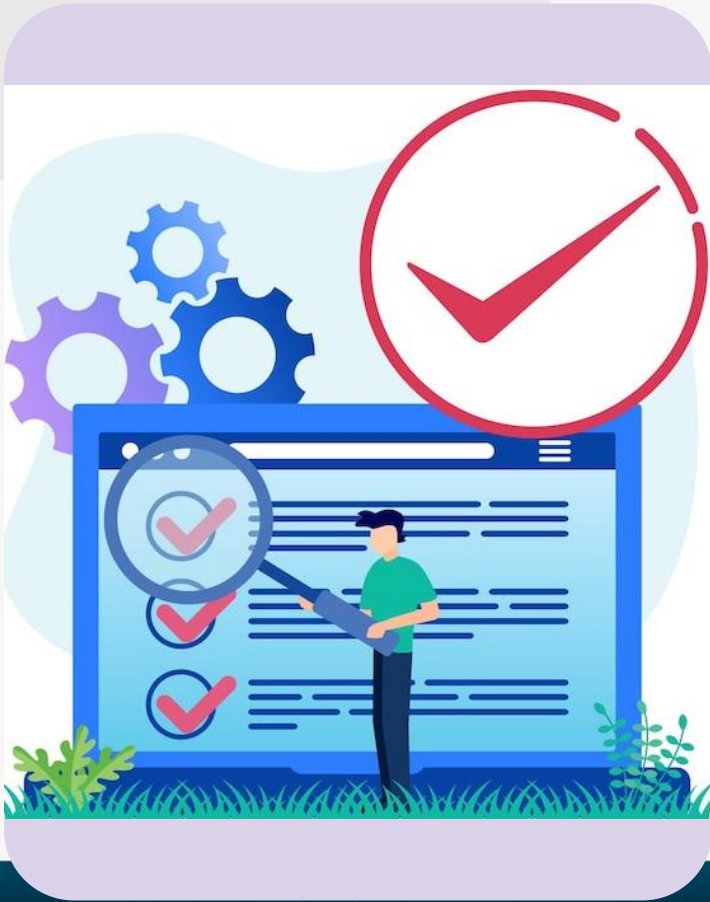
O modelo é dividido em duas partes:

- Qualidade do produto (interno/externo)
- Qualidade em uso



Qualidade do produto (interno/externo)

- **Adequação funcional (Functional Suitability):** O software faz o que precisa ?
- **Eficiência de desempenho (Performance Efficiency):** Consome poucos recursos ?
- **Compatibilidade (Compatibility):** Funciona com outros sistemas ?
- **Usabilidade (Usability):** É fácil e agradável de usar ?
- **Confiabilidade (Reliability):** Funciona sem falhas e se recupera bem de erros ?
- **Segurança (Security):** Protege dados e acessos ?
- **Manutenibilidade (Maintainability):** É fácil de corrigir e modificar ?
- **Portabilidade (Portability):** Pode ser executado em diferentes ambientes ?



Qualidade em uso

- Eficácia: o usuário consegue atingir seus objetivos ?
- Eficiência: o esforço para atingir os objetivos é razoável ?
- Satisfação: o usuário está satisfeito ?
- Ausência de riscos: o software evita riscos econômicos e de segurança ?
- Cobertura do contexto: o sistema se adapta a diferentes contextos de uso ?



Fatores de que Compõem a Qualidade

Confiabilidade



Manutenibilidade



Eficiência



Confiabilidade

É a capacidade do software de executar suas funções sob condições específicas por um período determinado.

Subfatores

- Maturidade (baixa taxa de falhas)
- Disponibilidade (tempo de funcionamento)
- Tolerância a falhas (continua operando mesmo com erros)
- Recuperabilidade (rapidez na restauração após falha)

Exemplo: sistema hospitalar que precisa operar 24h/dia, sem perder dados de pacientes.



Manutenibilidade

Refere-se à facilidade de corrigir defeitos, atualizar funções e adaptar o software a novos ambientes.

Subfatores

- Modularidade
- Reusabilidade
- Analisabilidade (facilidade de entender o código)
- Modificabilidade
- Testabilidade

Exemplo: um código bem documentado e com funções curtas e coesas é mais fácil de manter.



Eficiência

É a capacidade do software de utilizar recursos (tempo, memória, rede) de forma adequada.

Subfatores

- Comportamento temporal (tempo de resposta)
- Utilização de recursos (CPU, memória)
- Capacidade (número máximo de usuários simultâneos)

Exemplo: sites que não sobrecarregam o servidor mesmo com milhares de acessos simultâneos (como Google ou Netflix).

Testes de Software

Prof. Esp.
Rita de Cássia da Costa Silva



Testes de Software

Definição:

Processo sistemático de executar um sistema ou componente para identificar erros, defeitos ou comportamentos inesperados.

O objetivo não é provar que o software está certo, e sim garantir que ele funcione corretamente e detectar falhas.



NÍVEIS DE TESTE

Teste Unitário

Teste de Sistema

Teste de Integração

Teste de Aceitação

TESTE UNITÁRIO

Verifica unidades isoladas do código
(funções, métodos, classes).
Feito por desenvolvedores durante a codificação.

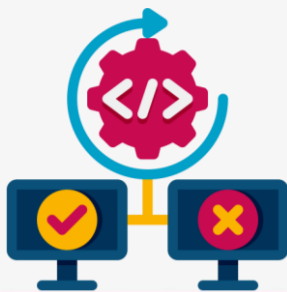
Exemplo: Testar uma função de
cálculo de desconto.

Ferramentas: JUnit (Java),
Pytest (Python), NUnit (.NET)



TESTE DE INTEGRAÇÃO

Verifica se os módulos interagem corretamente.



Exemplo: testar se o módulo de login comunica-se bem com o módulo de banco de dados.

Tipos: incremental, top-down, bottom-up.

TESTE DE SISTEMA

Testa o sistema completo, verificando se ele atende aos requisitos funcionais e não funcionais.



Obs: Inclui testes de desempenho, segurança e usabilidade.

TESTE DE ACEITAÇÃO

Realizado pelo cliente ou usuário final para validar se o software atende às suas necessidades.



- Baseia-se em casos de uso e requisitos.
- Pode envolver testes beta (em ambiente real com usuários).

Testes Manuais X Automatizados

TIPO	CARACTERÍSTICAS	VANTAGENS	DESVANTAGENS
Manuais	Feitos por pessoas, sem scripts automáticos	Flexibilidade, percepção humana	Mais lentos, sujeitos a erro humano
Automatizados	Scripts ou ferramentas executam os testes	Repetíveis, rápidos, ideais para regressão	Requerem mais tempo de configuração

Exemplo: Teste Unitário de um Software semelhante à uma calculadora na linguagem de programação Python

```
# Exemplo de teste unitário em Pytest
def soma(a, b):
    return a + b

def test_soma():
    assert soma(2, 3) == 5
```

“Os programadores não são aqueles que escrevem códigos perfeitos, mas sim aqueles que sabem como corrigir os erros.”

Barbara Liskov

E-mail: **rita.silva@faculdadefama.edu.br**

GitHub: **<https://github.com/ritasilva-fama>**